Contents

1 杂项 **1**
1.1 线下 vscode 配置 1
1.2 一些思路 1
1.3 大质数 及取模（常数小） 1
1.4 内存估算 2
1.5 正负数除法上下取整 2
1.6 对拍 2
1.7 int128 输入输出流 4
1.8 随机数 rng () 4
1.9 int128 输入输出 4
1.10 分数四则运算 4
1.11 基姆拉尔森公式 6
1.12 德州扑克 6
1.13 popcount 9
1.14 连续按位异或和 9
1.15 计算 n! 的末尾的 0 的个数 9
1.16 圆周率值 9
1.17 平方和公式展开恒等式 9
1.18 无序 (/指定顺序) 集合哈希 10
1.19 排列中 mex 与 min 的转化 10
1.20 Legendre 公式 10
1.21 O(1) 判断组合数奇偶性 11
1.22 图上重轻点根号分治 11

1 杂项

1.1 线下 vscode 配置

ctrl+ , 打开设置
搜 mouse 加上鼠标改编辑器和终端字体大小，
搜 smooth 把所有相关选项都打开 (美观)
自动编译运行 task, 按 ctrl+shift+B 找到 json 文件修改

```
{
    "label": "C++",
    "type": "shell",
    "command": "g++ -std=c++23 -O2 -pipe -static -s \"${file}\" -o exe && ./exe",
    "options": {
        "cwd": "${fileDirname}"
    },
    "presentation": {
        "reveal": "always",
        "focus": true,
        "panel": "shared"
    },
    "problemMatcher": "$gcc"
}
```

(改 ctrl+shift+n 的快捷键)
ctrl + shift+p 搜 open keyboard shortcuts(JSON)

```
[
    {
        "key": "Ctrl+Alt+n",
        "command": "workbench.action.tasks.runTask",
        "args": "C++"
    }
]
```

1.2 一些思路

- 贡献法: 考虑哪些元素会被计入答案
- 遇到 $a \bmod b$ 可以试着转化为 $a - b \times \lfloor \frac{a}{b} \rfloor$
- 值域比较小且数组中元素各不同可以考虑调和级数 nlogn 复杂度做法

1.3 大质数 及取模（常数小）

```
#define int long long
int P = (1LL<<61)-1;
```

```cpp
int mul(int a,int b){
    i128 x = (i128)a*b;
    u64 res = (u64)(x&P)+(u64)(x>>61);
    res = (res&P)+(res>>61);
    if(res>=P) res -= P;
    return (int)res;
}
```

1.4 内存估算

类型	字节数	256 MB 大约能开
`bool`	1 B	$2.68 * 10^8$
`char`	1 B	$2.68 * 10^8$
`short`	2 B	$1.34 * 10^8$
`int`	4 B	$6.71 * 10^7$
`long long`	8 B	$3.36 * 10^7$
`float`	4 B	$6.71 * 10^7$
`double`	8 B	$3.36 * 10^7$
`long double`	通常 16 B	$1.68 * 10^7$
指针	8 B（64 位）	$3.36 * 10^7$

1.5 正负数除法上下取整

```cpp
int Ceil(int fz,int fm){ // 2.5 -> 3, -2.1 -> -2
    if(fm<0) fm *= -1,fz *= -1;
    return fz>=0?(fz+fm-1)/fm:fz/fm;
}
int Floor(int fz,int fm){ // 2.5 -> 2 , -2.5 -> -3
    if(fm<0) fm *= -1,fz *= -1;
    return fz>=0?fz/fm:(fz-fm+1)/fm;
}
```

1.6 对拍

```bash
#!/bin/bash

g++ -std=c++2a -O2 test.cpp -o test
# g++ -std=c++2a -O2 check.cpp -o check
g++ -std=c++2a -O2 std.cpp -o std
g++ -std=c++2a -O2 data.cpp -o data

// 普通对拍
for((i=1;;i++))
do
    ./data > in.txt
    ./test < in.txt > out_user.txt
    ./std  < in.txt > out_brute.txt
    if ! diff out_user.txt out_brute.txt > /dev/null
    then
        echo "发现不一致！输入如下："
        cat in.txt
        echo "test 输出："
        cat out_user.txt
        echo "std 输出："
        cat out_brute.txt
        exit
    fi
    echo "通过测试 $i"
done
// 带check的
for((i=1;;i++))
do
    ./data > in.txt
    cat in.txt > temp.txt
    ./test < in.txt >> temp.txt
    ./check < temp.txt > out_user.txt
    ./std  < in.txt > out_brute.txt
    if ! diff out_user.txt out_brute.txt > /dev/null
    then
        echo "发现不一致！输入如下："
        cat in.txt
        echo "test 输出："
        cat out_user.txt
```

```
        echo "std 输出: "
        cat out_brute.txt
        exit
    fi
    echo "通过测试 $i"
done
```

1.7 int128 输入输出流

```
using i128 = __int128;

// 重载输入运算符以支持__int128类型
std::istream &operator>>(std::istream &is, __int128 &val) {
    std::string s;
    bool flag = true;
    is >> s, val = 0;
    if (s[0] == '-') flag = false, val = std::stoi(s.substr(0, 2)), s = s.substr(2);
    for (char &c: s) val = val * 10 + (c - '0') * (!flag ? -1 : 1);
    return is;
}

//重载输出运算符以支持__int128类型
std::ostream &operator<<(std::ostream &os, __int128 val) {
    if (val < 0) os << "-", val = -val;
    if (val > 9) os << val / 10;
    os << static_cast<char>(val % 10 + '0');
    return os;
}
```

1.8 随机数 rng ()

```
std::mt19937_64 rng(std::chrono::system_clock::now().time_since_epoch().count());
```

1.9 int128 输入输出

赛时打嫌浪费时间可以 #define int __int128, 然后每次输入 (long long)t 然后赋值给指定的变量，最后输出结果的时候在 (long long)ans (((

```
using i128 = __int128;

// 重载输入运算符以支持__int128类型
std::istream &operator>>(std::istream &is, __int128 &val) {
    std::string s;
    bool flag = true;
    is >> s, val = 0;
    if (s[0] == '-') flag = false, val = std::stoi(s.substr(0, 2)), s = s.substr(2);
    for (char &c: s) val = val * 10 + (c - '0') * (!flag ? -1 : 1);
    return is;
}

//重载输出运算符以支持__int128类型
std::ostream &operator<<(std::ostream &os, __int128 val) {
    if (val < 0) os << "-", val = -val;
    if (val > 9) os << val / 10;
    os << static_cast<char>(val % 10 + '0');
    return os;
}
```

1.10 分数四则运算

```
template<class T>
struct Frac {
    T num;  //分子
    T den;  // 分母

    Frac(T num_, T den_) : num(num_), den(den_) {
        if (den < 0) {
            den = -den;
            num = -num;
        }
    }

    Frac() : Frac(0, 1) {}

    Frac(T num_) : Frac(num_, 1) {}

    explicit operator double() const {
        return 1. * num / den;
    }

    Frac &operator+=(const Frac &rhs) {
        num = num * rhs.den + rhs.num * den;
        den *= rhs.den;
        return *this;
    }

    Frac &operator-=(const Frac &rhs) {
        num = num * rhs.den - rhs.num * den;
```

```
    den *= rhs.den;
    return *this;
}

Frac &operator*=(const Frac &rhs) {
    num *= rhs.num;
    den *= rhs.den;
    return *this;
}

Frac &operator/=(const Frac &rhs) {
    num *= rhs.den;
    den *= rhs.num;
    if (den < 0) {
        num = -num;
        den = -den;
    }
    return *this;
}

friend Frac operator+(Frac lhs, const Frac &rhs) {
    return lhs += rhs;
}

friend Frac operator-(Frac lhs, const Frac &rhs) {
    return lhs -= rhs;
}

friend Frac operator*(Frac lhs, const Frac &rhs) {
    return lhs *= rhs;
}

friend Frac operator/(Frac lhs, const Frac &rhs) {
    return lhs /= rhs;
}

friend Frac operator-(const Frac &a) {
    return Frac(-a.num, a.den);
}

friend bool operator==(const Frac &lhs, const Frac &rhs) {
    return lhs.num * rhs.den == rhs.num * lhs.den;
}

friend bool operator!=(const Frac &lhs, const Frac &rhs) {
    return lhs.num * rhs.den != rhs.num * lhs.den;
}

friend bool operator<(const Frac &lhs, const Frac &rhs) {
    return lhs.num * rhs.den < rhs.num * lhs.den;
}

friend bool operator>(const Frac &lhs, const Frac &rhs) {
    return lhs.num * rhs.den > rhs.num * lhs.den;
}

friend bool operator<=(const Frac &lhs, const Frac &rhs) {
    return lhs.num * rhs.den <= rhs.num * lhs.den;
}

friend bool operator>=(const Frac &lhs, const Frac &rhs) {
    return lhs.num * rhs.den >= rhs.num * lhs.den;
}

friend std::ostream &operator<<(std::ostream &os, Frac x) {
    T g = std::gcd(x.num, x.den);
    if (x.den == g) {
        return os << x.num / g;
    } else {
        return os << x.num / g << "/" << x.den / g;
    }
}

std::string to_string() {
    T g = std::gcd(num, den);
    if (den == g) {
        return std::to_string(num) + "/" + std::to_string(g);
    } else {
        return std::to_string(num / g) + "/" + std::to_string(den / g);
    }
```

```
    }
};
```

1.11 基姆拉尔森公式

```
/**   基姆拉尔森公式（计算某年某月某日是星期几）
 *    2023-09-05: https://qoj.ac/submission/164735
**/
const int d[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

bool isLeap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}

int daysInMonth(int y, int m) {
    return d[m - 1] + (isLeap(y) && m == 2);
}

int getDay(int y, int m, int d) {
    int ans = 0;
    for (int i = 1970; i < y; i++) {
        ans += 365 + isLeap(i);
    }
    for (int i = 1; i < m; i++) {
        ans += daysInMonth(y, i);
    }
    ans += d;
    return (ans + 2) % 7 + 1;
}
```

1.12 德州扑克

```
namespace holdem {
    enum Suit {
        HighCard,           // 高牌
        Pair,               // 对子
        TwoPair,            // 两对
        ThreeOfAKind,       // 三条
        Straight,           // 顺子
        Flush,              // 同花
        FullHouse,          // 葫芦
        FourOfAKind,        // 铁支
        StraightFlush,      // 同花顺
        RoyalStraightFlush  // 同花大顺
    };
    enum CardType {
        Two = 0,
        Three = 1,
        Four = 2,
        Five = 3,
        Six = 4,
        Seven = 5,
        Eight = 6,
        Nine = 7,
        Ten = 8,
        Jack = 9,
        Queen = 10,
        King = 11,
        Ace = 12
    };
    enum Color {
        Heart = 0,    //
        Spade = 1,    //
        Club = 2,     //
        Diamond = 3   //
    };
    // 一张卡牌，用类型和颜色表示
    using Card = std::pair<CardType, Color>;
    bool operator<(const Card &lhs, const Card &rhs) {
        return lhs.first < rhs.first;
    }
    // 手牌分数，用牌型和分数判断，分数只对相同牌型有效
    using Score = std::pair<Suit, int>;
    bool operator<(const Score &lhs, const Score &rhs) {
        return lhs.first < rhs.first or (lhs.first == rhs.first and lhs.second < rhs.second);
    }
    // 手牌
    struct Hand {
        // 颜色，用位表示每张手牌是否存在
```

```
        std::array<short, 4> color{};
        // 数目，存放每种卡牌的数目
        std::array<short, 13> type{};
        void clear(){
            std::fill(color.begin(), color.end(), 0);
            std::fill(type.begin(), type.end(), 0);
        }
        void addCard(const Card& c){
            color[c.second] += (1 << c.first);
            type[c.first]++;
        }
        void removeCard(const Card& c){
            color[c.second] -= (1 << c.first);
            type[c.first]--;
        }
    };
    Score getType(const Hand &hand) {

        auto& colors = hand.color;
        auto& types = hand.type;
        // is RoyalStraightFlush
        for (short i = 0; i < 4; ++i) {
            if ((colors[i] & 0b1111110000000000) == 0b1111100000000000) {
                return {RoyalStraightFlush, 0};
            }
        }
        // is StraightFlush
        {
            int tv = 0b1111100000000000;
            for (short i = Ace; i >= Five; --i) {
                for(int j = 0; j < 4; ++j){
                    if((colors[j] & tv) == tv){
                        return {StraightFlush, i};
                    }
                }
                tv >>= 1;
                if(__builtin_popcount(tv) < 5) tv |= (1 << 12);
            }
        }
        // is FourOfAKind
        {
            for (short i = Ace; i >= Two; --i) {
                if (types[i] >= 4) {
                    for (short j = Ace; j >= Two; --j) {
                        if (j == i) continue;
                        if (types[j] > 0) {
                            return {FourOfAKind, i * 13 + j};
                        }
                    }
                }
            }
        }
        // is FullHouse
        {
            for (short i = Ace; i >= Two; --i) {
                if (types[i] >= 3) {
                    for (short j = Ace; j >= Two; --j) {
                        if (j == i) continue;
                        if (types[j] >= 2) {
                            return {FullHouse, i * 13 + j};
                        }
                    }
                }
            }
        }
        // is Flush
        {
            int score = 0;
            for (short c = 0; c < 4; ++c) {
                if (__builtin_popcount(colors[c]) >= 5) {
                    int tScore = 0;
                    for (int j = Ace; j >= Two and __builtin_popcount(tScore) < 5; --j) {
                        tScore |= (colors[c] & (1 << j));
                    }
                    score = std::max(score, tScore);
                }
            }
            if (score) return {Flush, score};
        }
        // is Straight
```

```
        {
            for (int i = Ace; i >= Five; --i) {
                bool ok = true;
                for (int j = 0; j < 5 and ok; ++j) {
                    int p = (i + 13 - j) % 13;
                    if (types[p] == 0) {
                        ok = false;
                    }
                }
                if (ok) {
                    return {Straight, i};
                }
            }
        }
        // is ThreeOfAKind
        {
            for (int i = Ace; i >= Two; --i) {
                if (types[i] >= 3) {
                    std::vector<int> others;
                    for (int j = Ace; j >= Two and others.size() < 2; --j) {
                        if (types[j] > 0) {
                            others.push_back(j);
                        }
                    }
                    return {ThreeOfAKind, i * 169 + others[0] * 13 + others[1]};
                }
            }
        }
        // is TwoPair
        {
            std::vector<int> pairs;
            for (int i = Ace; i >= Two and pairs.size() < 2; --i) {
                if (types[i] >= 2) {
                    pairs.push_back(i);
                }
            }
            if (pairs.size() >= 2) {
                for (int i = Ace; i >= Two; --i) {
                    if (types[i] > 0) {
                        return {TwoPair, pairs[0] * 169 + pairs[1] * 13 + i};
                    }
                }
            }
        }
        // is Pair
        {
            for (int i = Ace; i >= Two; --i) {
                if (types[i] >= 2) {
                    int score = i;
                    for (int j = Ace, cnt = 0; j >= Two and cnt < 3; --j) {
                        if (j == i) continue;
                        if (types[j] > 0) {
                            ++cnt;
                            score = score * 13 + j;
                        }
                    }
                    return {Pair, score};
                }
            }
        }
        // HighCard
        int highScore = 0;
        for (int i = Ace, cnt = 0; i >= Two and cnt < 5; --i) {
            if (types[i]) {
                highScore = highScore * 13 + i;
                ++cnt;
            }
        }
        return {HighCard, highScore};
    }
    char cardType2char(CardType ct) {
        return "23456789TJQKA"[ct];
    }
    CardType char2cardType(char c) {
        switch (c) {
            case '2':
                return Two;
            case '3':
                return Three;
            case '4':
```

```
                return Four;
            case '5':
                return Five;
            case '6':
                return Six;
            case '7':
                return Seven;
            case '8':
                return Eight;
            case '9':
                return Nine;
            case 'T':
                return Ten;
            case 'J':
                return Jack;
            case 'Q':
                return Queen;
            case 'K':
                return King;
            default:
                return Ace;
        }
    }
}
```

整理自 www.algowiki.cn/competitions?tab=tricks

1.13 popcount

```
__builtin_popcount(x); // int
__builtin_popcountll(x); // long long
```

1.14 连续按位异或和

设

$$S(n) = 1 \oplus 2 \oplus 3 \oplus \cdots \oplus n$$

则有：

$$S(n) = \begin{cases} n, & n \bmod 4 = 0 \\ 1, & n \bmod 4 = 1 \\ n+1, & n \bmod 4 = 2 \\ 0, & n \bmod 4 = 3 \end{cases}$$

1.15 计算 n! 的末尾的 0 的个数

公式如下：

$$ans = \lfloor \frac{n}{5} \rfloor + \lfloor \frac{n}{25} \rfloor + \lfloor \frac{n}{125} \rfloor + \ldots$$

计算 $n!$ 的末尾 0 的个数，实际上就是计算 $1 * 2 * 3 * \ldots * n - 1 * n$ 中因子 10 的个数
$10 = 2 * 5$，且在 $n!$ 中，因子 2 的个数远远多于因子 5 的个数，所以只需要统计 $n!$ 中所有因子 5 的个数即可
已知 1 到 n 中 5 的倍数的个数为 $\lfloor \frac{n}{5} \rfloor$，但如果直接将 $\lfloor \frac{n}{5} \rfloor$ 当作答案会漏掉 5 的个数 因为 $25 = 5 * 5$ 贡献了 2 个 5，$125 = 5 * 5 * 5$ 贡献了 3 个 $5 \cdots$
可以通过重复计数来补上这些漏掉的 5 - $\lfloor \frac{n}{5} \rfloor$ 统计所有 5 的倍数贡献的第一个 5 - $\lfloor \frac{n}{25} \rfloor$ 统计所有 25 的倍数贡献的第二个 5 - $\lfloor \frac{n}{125} \rfloor$ 统计所有 125 的倍数贡献的第三个 5 - ……
这样一来就可以完整地统计出 $n!$ 中所有因子 5 的个数了

1.16 圆周率值

```
using f64 = long double;
const f64 pi = acosl(-1);
```

1.17 平方和公式展开恒等式

基于**平方和**与**和平方**之间的恒等式

$$(\sum_{i=1}^{n} s_i)^2 = \sum_{i=1}^{n} s_i^2 + \sum_{i \neq j} s_i s_j$$

容易发现：

$$\sum_{i \neq j} s_i s_j = 2 \sum_{1 \leq i < j \leq n} s_i s_j$$

所以有：

$$\sum_{1\le i<j\le n} s_i s_j = \frac{(\sum_{i=1}^{n} s_i)^2 - \sum_{i=1}^{n} s_i^2}{2}$$

应用：数组两两之间的乘积和（$O(n^2)$ 降为 $O(n)$）
普通求 $\sum_{1\le i<j\le n} s_i s_j$ 直接利用前缀和就可以做到 $O(n)$
但是如果需要**区间查询 + 带修**就可以用这个恒等式用线段树来简单实现
线段树维护**区间查询 + 带修** 也可以不使用这个恒等式，每个节点维护：

- `sum`：区间元素和。
- `ans`：区间两两乘积之和。

合并左右儿子：

$$sum = sum_L + sum_R$$
$$ans = ans_L + ans_R + sum_L \cdot sum_R$$

(当然利用前缀和 ddp 也可以，不过会增加代码实现难度)

1.18 无序 (/指定顺序) 集合哈希

判断两个集合是否相同，使用满足交换律的运算 $\sum rng(a[i])$, $\prod$,xor $\cdots$. 等操作哈希一下就行 (注意避免哈希冲突)
区间查询实现

- 不带修，使用前缀和维护，单次查询时间复杂度：$O(1)$。
- 带修，使用数据结构（如树状数组/线段树）维护区间和，单次查询时间复杂度：$O(\log n)$。

扩展问题（指数哈希）
给定数组 a，多次询问：判断区间 $[L_1, R_1]$ 中所有元素加上 k 后，是否与区间 $[L_2, R_2]$ 的可重集合相等。
设常数 B，定义哈希函数：$H(S) = \sum B^{a_i}$
则有：$H([L, R]) = \sum_{i=L}^{R} B^{a_i}$
区间整体加上 k 后：$H'([L, R]) = \sum_{i=L}^{R} B^{a_i+k} = B^k \cdot \sum_{i=L}^{R} B^{a_i} = B^k \cdot H([L, R])$
所以仅需要判断 $B^k \cdot H([L_1, R_1])$ 与 $H([L_2, R_2])$ 的关系即可。

1.19 排列中 mex 与 min 的转化

对于一个排列，前缀 mex 的询问，可以转换为后缀最小值：

$$\mathrm{mex}_{i=1}^{j} a_i = \min_{i=j+1}^{n} a_i$$

约数个数奇偶性
一个数的约数个数是**奇数**等价于这个数是**完全平方数**。

1.20 Legendre 公式

记 $v_2(x)$ 为 x 中质因数 2 的指数, 那么

$$v_2(n!) = \sum_{k\ge 1} \left\lfloor \frac{n}{2^k} \right\rfloor = n - \mathrm{popcount}(n)$$

推导：
设 n 的二进制表示为：

$$n = \sum_{i=0}^{k} b_i 2^i, \quad b_i \in \{0, 1\}$$

关键是把每一项的 n 展开。

$$\begin{aligned}\sum_{j\geq 1}\left\lfloor\frac{n}{2^j}\right\rfloor &= \sum_{j\geq 1}\sum_{i\geq j} b_i 2^{i-j}\\ &= \sum_{i\geq 1} b_i \sum_{j=1}^{i} 2^{i-j}\\ &= \sum_{i\geq 1} b_i (2^i - 1)\\ &= \sum_{i\geq 1} b_i 2^i - \sum_{i\geq 1} b_i\\ &= n - \text{popcount}(n)\end{aligned}$$

1.21 O(1) 判断组合数奇偶性

$\binom{n}{m}$ 为奇数 $\Leftrightarrow (m\&(n-m)) = 0$

上面的方法可以 $O(1)$ 判断, 如果使用 lucas 定理的话可以 $O(\log)$ 复杂度求

推导:

因为 $\binom{n}{m} = \frac{\prod_{m+1}^{n} i}{\prod_{1}^{m} i}$, 那么 $\binom{n}{m}$ 是不是奇数就是看 $\prod_{m+1}^{n} i$ 的质因数 2 有没有被 $\prod_{1}^{m} i$ 消尽

记 $v_2(x)$ 为 x 中质因数 2 的指 (个) 数，那么

$$v_2\binom{n}{m} = v_2(n!) - v_2(m!) - v_2((n-m)!)$$

根据 Legendre 公式：

$$v_2(n!) = \sum_{k\geq 1}\left\lfloor\frac{n}{2^k}\right\rfloor = n - \text{popcount}(n)$$

其中 popcount(n) 表示 n 的二进制中 1 的个数。代入可得：

$$\boxed{v_2\binom{n}{m} = \text{popcount}(m) + \text{popcount}(n-m) - \text{popcount}(n)}$$

$v_2\binom{n}{m} == 0$ 即

$$\text{popcount}(m) + \text{popcount}(n-m) = \text{popcount}(n)$$

$m + (n-m) = n$, 两个二进制数相加，只有不发生进位时，二进制中 1 的总数才不会减少。

也就是 n 和 n-m 两个数的二进制中，没有同一位同时为 1 , 故 $(m\&(n-m)) = 0$

1.22 图上重轻点根号分治

设阈值 $B = \sqrt{m}$。

- 轻点：deg$(u) \leq B$。邻居最多 $O(\sqrt{m})$，可以暴力枚举。
- 重点：deg$(u) > B$。因为所有点的度数和为 $2m$，重点最多 $O(\sqrt{m})$ 个，可以针对重点预处理或维护信息。

在具有 m 条边的图中，最多只有 $O(\sqrt{m})$ 种度数不同的结点。